Other useful built-in functions that complement `count()` are `is_array()` (tests if a variable is an array), and `is_countable()` (tests if a variable can be counted, which includes arrays and `ArrayObject`).

With `for`

PHP, like many other languages, provides the construct `for`. This construct is designed to execute a block of code in a loop until something causes it to exit and return control to the main program. It is more complex than `foreach` in that it usually requires some foreknowledge about the array being processed. The `for` statement requires three expressions, which are placed in parentheses immediately following the keyword like this: `for (expr_begin ; expr_before ; expr_after) { //DO SOMETHING }`.

The three expressions are separated with semicolons; the first one is evaluated before the loop begins; the second one is evaluated at the beginning of each iteration, and if `true`, then the block executes; and the third one is evaluated after each iteration. Let's see it in action:

```
$myArr = ['Alice', 'Rita', 'Sue', 'Bob'];
$size = count($myArr);

for ($i = 0; $i < $size; $i++)  {
    print "{$myArr[$i]} is here.\n";
}

print "\$i = {$i}\n";

Output:
Alice is here.
Rita is here.
Sue is here.
Bob is here.
$i = 4
```

This is a fairly typical pattern. The first expression – `$i = 0;` – initializes the variable `$i` to the integer zero, which is used as the key to access the elements of `$myArr` inside the loop's code block. The second checks that `$i` isn't larger than the maximum index of